# Введение

В современном мегаполисе, таком как Москва, здоровый образ жизни и регулярные занятия спортом становятся неотъемлемой частью повседневной культуры горожан. По данным Департамента спорта города Москвы, в столице функционирует более 800 спортивных объектов различной направленности: от крупных стадионов и дворцов спорта до районных спортивных комплексов и воркаут-площадок. Однако наличие инфраструктуры само по себе не решает проблему её эффективного использования гражданами.

Основная проблема заключается в сложности поиска подходящего спортивного объекта, удовлетворяющего индивидуальным критериям пользователя. Потенциальный посетитель сталкивается с необходимостью учитывать множество факторов: местоположение, перечень доступных видов спорта, график работы, а также наличие условий для людей с ограниченными возможностями здоровья. Существующие решения, такие как официальные сайты спортивных комплексов или портал открытых данных Правительства Москвы data.mos.ru, предоставляют информацию в сыром, неструктурированном виде, требуя от пользователя дополнительных усилий по её анализу и сопоставлению. Картографические сервисы, например Яндекс.Карты, позволяют оценить местоположение, но не предоставляют детальной фильтрации по видам спорта или параметрам доступной среды.

В рамках данного курсового проекта разработано веб-приложение «Спорт рядом», призванное решить обозначенную проблему. Приложение обеспечивает удобный поиск спортивных объектов Москвы с возможностью фильтрации по названию, видам спорта, району и доступности для маломобильных групп населения. Реализованы функции добавления объектов в избранное, их сравнения по ключевым характеристикам, а также поиск ближайших объектов относительно заданного пользователем адреса. В качестве источника данных использован открытый датасет спортивных объектов с портала data.mos.ru, насчитывающий 810 записей.

# Цель и задачи работы

Целью данной работы является разработка веб-приложения для поиска спортивных объектов Москвы, учитывающего индивидуальные потребности пользователей и обеспечивающего удобный доступ к открытым данным.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ открытых источников данных и выбрать подходящий датасет;
- выполнить предобработку и очистку данных для дальнейшего использования;
- спроектировать структуру базы данных для хранения информации о спортивных объектах и пользователях;
- разработать серверную часть приложения на языке Python с использованием фреймворка Flask;
- создать пользовательский интерфейс с помощью HTML и CSS;
- реализовать функционал фильтрации, поиска, избранного и сравнения объектов;
- реализовать поиск ближайших объектов по заданному адресу с использованием геокодинга;
- провести тестирование и отладку разработанного приложения.

В качестве исходных данных используется датасет «Спортивные объекты города Москвы» с портала открытых данных Правительства Москвы data.mos.ru. Датасет содержит 810 записей и включает информацию о названиях объектов, адресах, видах спорта, графике работы, контактах и параметрах доступной среды.

Для реализации проекта выбраны следующие средства разработки:

- Python 3.11 — язык программирования для серверной части;
- Flask — веб-фреймворк для создания приложения;
- PostgreSQL — система управления базами данных;
- HTML, CSS — для создания пользовательского интерфейса;
- Git — система контроля версий;
- Nominatim (OpenStreetMap) — для геокодинга адресов.

# Проектирование приложения

На этапе проектирования были определены основные функциональные требования к приложению, сформулированные в виде пользовательских историй. Пользовательские истории позволяют описать потребности целевой аудитории и необходимый функционал с точки зрения пользователя.

Первая пользовательская история описывает потребность в поиске объектов по различным критериям: пользователь хочет находить спортивные объекты по названию, виду спорта, району и доступности для маломобильных групп населения. Это позволит быстро отбирать подходящие варианты без необходимости просматривать все объекты.

Вторая пользовательская история связана с получением детальной информации об объекте. Пользователь, выбрав конкретный объект из списка, должен иметь возможность просмотреть полную информацию: точный адрес, часы работы, контактные данные, перечень доступных видов спорта, параметры доступной среды, а также увидеть местоположение объекта на карте.

Третья пользовательская история предполагает возможность сравнения нескольких объектов. Пользователь может выбрать от двух до трёх объектов и увидеть их ключевые характеристики в виде таблицы, что облегчает процесс выбора наиболее подходящего варианта.

Четвёртая пользовательская история связана с персонализацией приложения. Зарегистрированный пользователь может добавлять понравившиеся объекты в избранное для быстрого доступа к ним в будущем, а также указывать свой домашний адрес.

Пятая пользовательская история развивает идею персонализации: пользователь, указавший домашний адрес, может получить список спортивных объектов, расположенных ближе всего к его дому, с сортировкой по расстоянию.

На этапе проектирования были разработаны макеты основных страниц приложения. На рисунке 1 представлен макет главной страницы. На макете показана панель фильтров, которая может выдвигаться, и список объектов в виде карточек. Стрелками обозначены возможные пути навигации: применение фильтров, переход к детальной странице, добавление объектов в сравнение.

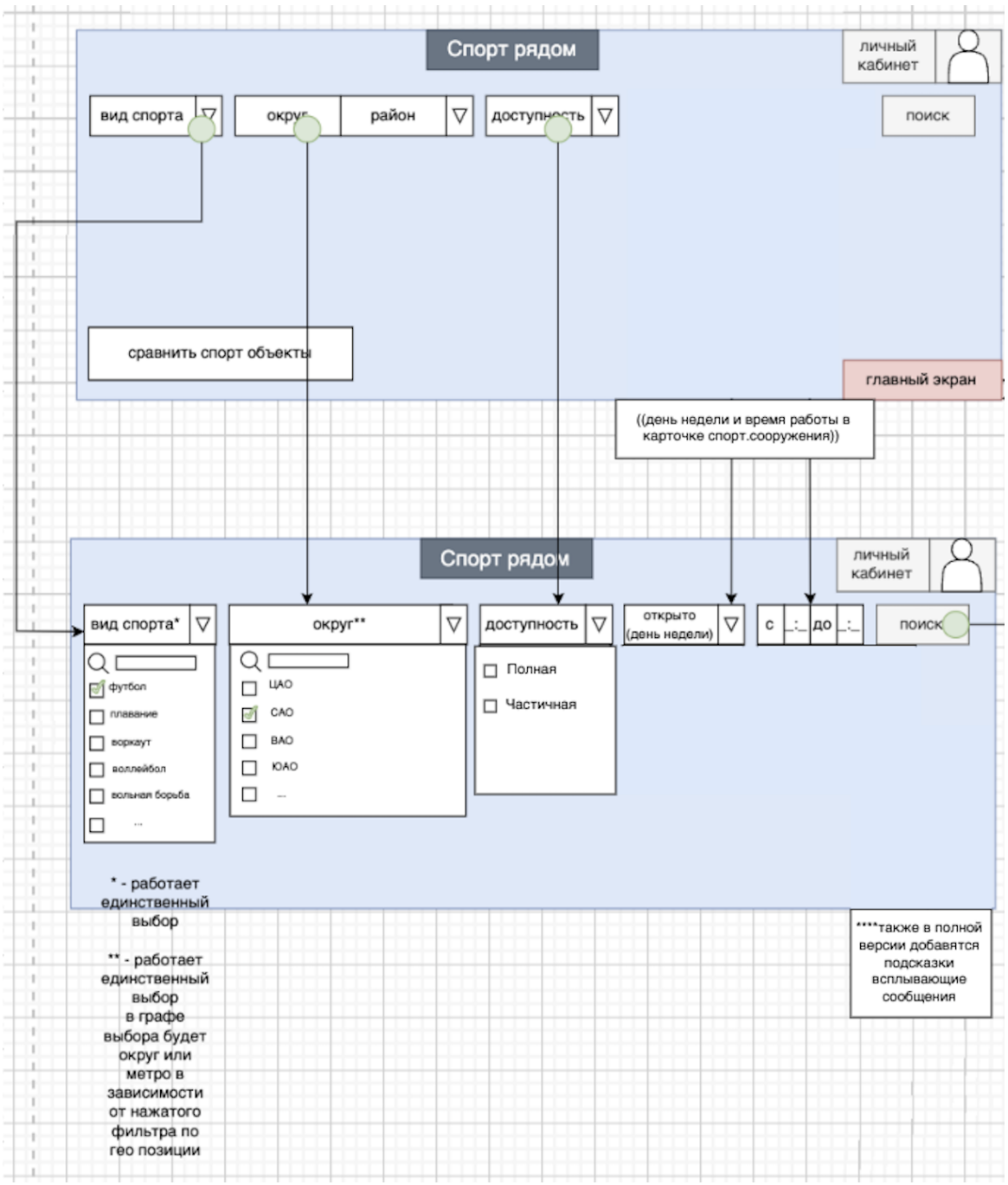


Рис. 1: Макет главной страницы

На рисунке 2 показан макет детальной страницы объекта. На нём отражена структура расположения информации: название объекта, виды спорта, параметры доступной среды, часы работы, контакты и карта. Стрелка демонстрирует переход с главной страницы на детальную после нажатия кнопки поиска.

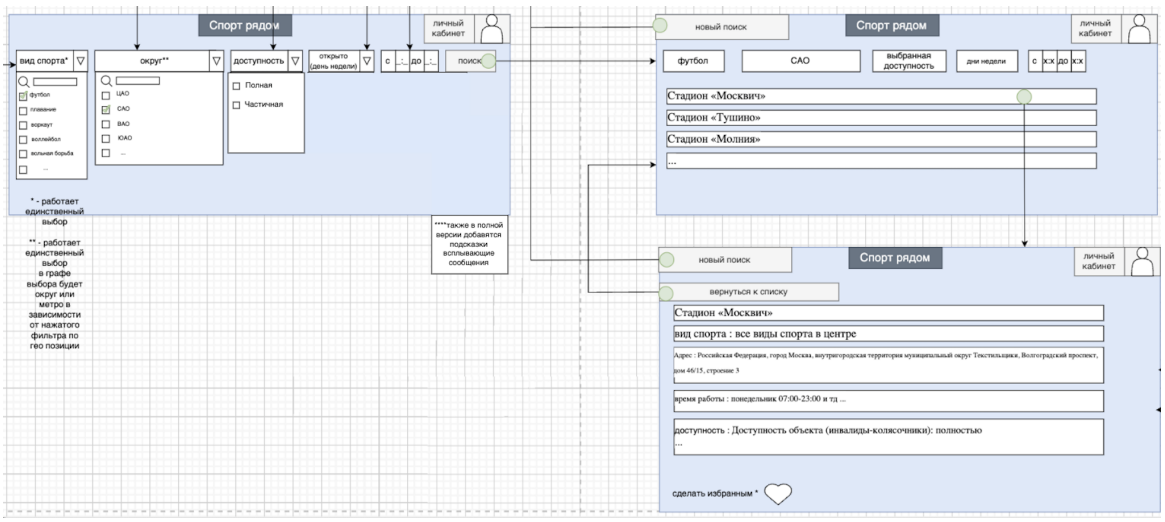


Рис. 2: Макет детальной страницы

На рисунке 3 представлен макет личного кабинета. Стрелкой показан переход с главной страницы в личный кабинет. На макете отображены имя пользователя, фамилия, список избранных объектов.

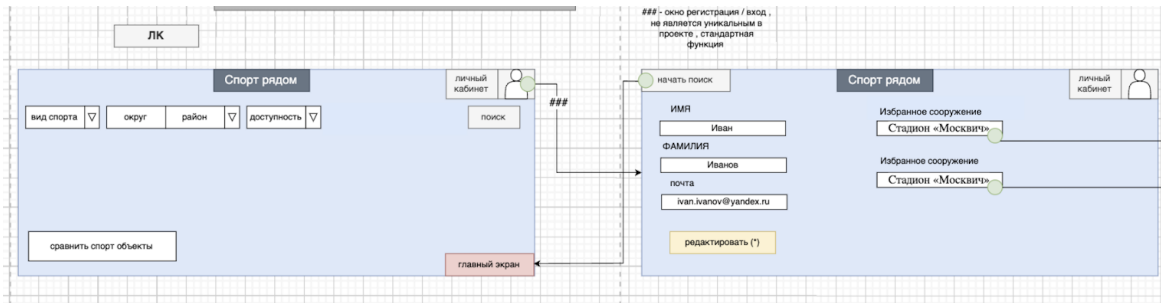


Рис. 3: Макет личного кабинета

## Проектирование базы данных

На основе сформулированных требований была спроектирована структура базы данных, включающая три таблицы.

Таблица `sport_facilities` хранит информацию о спортивных объектах: название, адрес, административный округ и район, перечень видов спорта (в виде массива), параметры доступной среды для четырёх категорий маломобильных групп населения, часы работы, контактные данные, географические координаты и уникальный идентификатор из исходного датасета.

Таблица `users` содержит данные зарегистрированных пользователей: имя, фамилию, email (используется для входа), хэш пароля, домашний адрес и дату регистрации.

Таблица `favorites` является связующей и реализует отношение многие-ко-многим между пользователями и спортивными объектами. Она содержит идентификаторы пользователя и объекта, а также временную метку добавления. Составной первичный ключ `(user_id, facility_id)` предотвращает повторное добавление одного и того же объекта одним пользователем. Внешние ключи с каскадным удалением обеспечивают согласованность данных: при удалении пользователя или объекта все связанные записи в избранном удаляются автоматически.

Спроектированная структура базы данных была визуализирована с помощью инструмента DBeaver и представлена на рисунке 4. На диаграмме показаны три таблицы и связи

между ними.

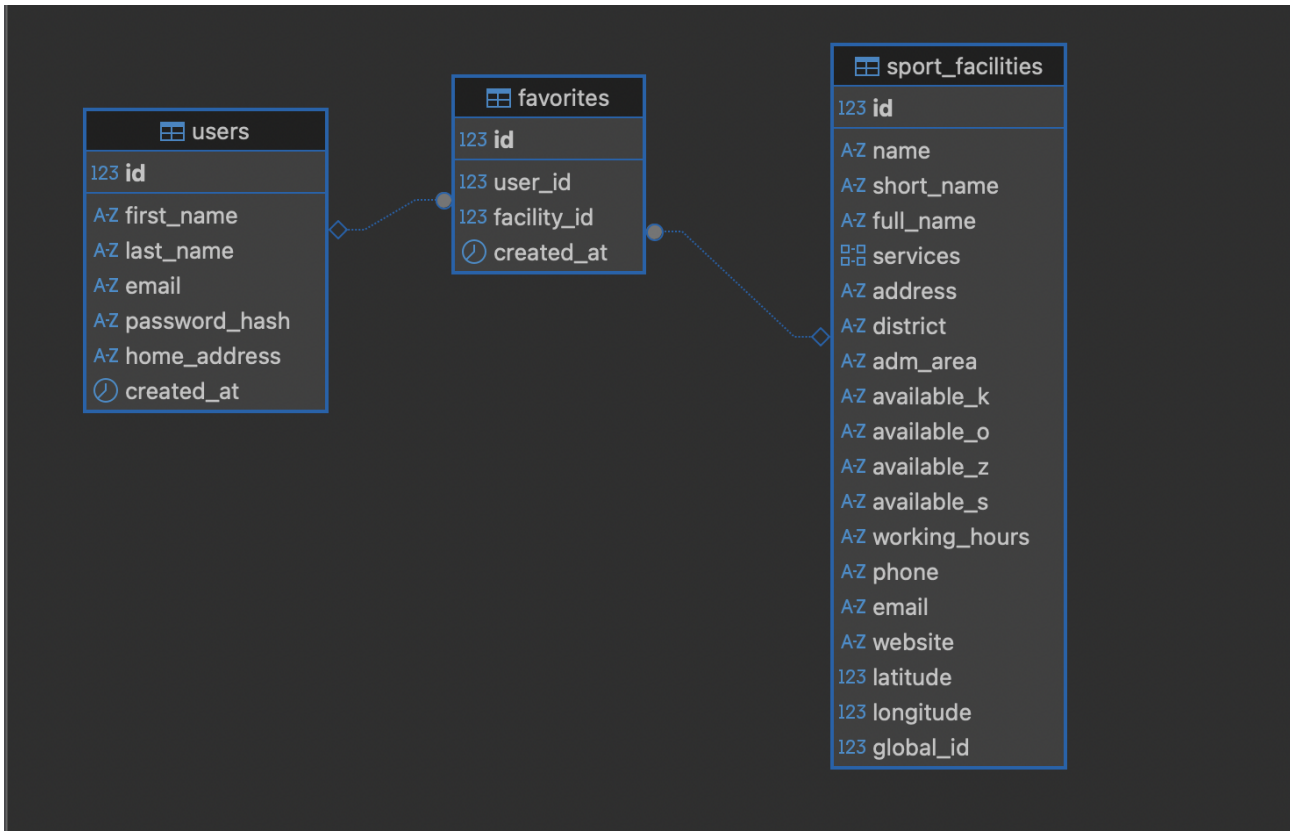


Рис. 4: ER-диаграмма базы данных sport_ryadom (визуализация в DBeaver)

## Проектирование интерфейса

До начала разработки были созданы макеты основных страниц приложения, что позволило визуализировать структуру интерфейса и согласовать расположение элементов.

На главной странице размещены следующие функциональные блоки:

- панель навигации с ссылками на вход/регистрацию или личный кабинет;
- блок фильтров, содержащий поля для поиска по названию, выпадающие списки для выбора вида спорта, доступности и района;
- кнопка применения фильтров;
- кнопка сравнения выбранных объектов;
- список объектов, представленных в виде карточек, каждая из которых содержит название, адрес, район, информацию о доступности и флажок для добавления в сравнение.

Детальная страница объекта включает:

- название объекта и кнопку добавления в избранное;
- подробную информацию о расположении (адрес, район);
- перечень доступных видов спорта в виде цветных тегов;
- таблицу с параметрами доступной среды;

- график работы;
- контактные данные (телефон, email);
- карту с меткой местоположения объекта.

Личный кабинет содержит:

- имя пользователя;
- информацию об email;
- форму для ввода и сохранения домашнего адреса;
- кнопку для поиска ближайших объектов;
- список избранных объектов со ссылками на их детальные страницы.

Страница сравнения представляет собой таблицу, в строках которой перечислены характеристики объектов, а в столбцах — сами объекты. Такое представление позволяет наглядно сопоставить параметры нескольких объектов и выбрать наиболее подходящий.

Страница выбора объектов для сравнения открывается, если пользователь отметил более трёх объектов на главной. На этой странице все выбранные объекты отображаются с установленными флажками, и пользователь может снять лишние, оставив от двух до трёх для сравнения. При отправке формы серверная логика на Python проверяет количество выбранных объектов: если выбрано менее двух, пользователь возвращается на страницу выбора с сообщением об ошибке; если выбрано более трёх — снова показывается страница выбора с исходным списком объектов.

Страница с результатами геопоиска содержит список объектов, отсортированных по расстоянию от указанного пользователем адреса. Для каждого объекта отображается расстояние в километрах, название, адрес, район и информация о доступности.

### Проектирование архитектуры

Архитектура приложения построена по принципу клиент-серверного взаимодействия. Серверная часть, реализованная на Flask, обрабатывает HTTP-запросы, выполняет бизнес-логику и взаимодействует с базой данных. Клиентская часть представляет собой набор HTML-страниц с минимальным использованием CSS для стилизации. Взаимодействие между клиентом и сервером осуществляется синхронно: при выполнении действий пользователя страница перезагружается с обновлёнными данными.

Такая архитектура была выбрана из соображений простоты разработки и отладки, а также для обеспечения совместимости с любыми современными браузерами без необходимости использования сложных JavaScript-фреймворков.

## Реализация приложения

Реализация приложения началась с этапа предобработки исходных данных. Датасет в формате XLSX содержал информацию в структурированном, но неудобном для непосредственного использования виде. В частности, поля с услугами содержали перечень видов спорта в квадратных скобках, поля с адресами включали вложенную информацию об округе, районе и параметрах доступности, а поле с часами работы содержало многострочный текст с английскими обозначениями дней недели.

Для загрузки и предобработки данных был разработан отдельный скрипт load_data.py, использующий библиотеки pandas и psycopg2. Скрипт выполняет чтение XLSX-файла, парсинг вложенных структур и загрузку очищенных данных в таблицу sport_facilities базы PostgreSQL. В процессе загрузки выполняются следующие преобразования:

- из строки с услугами удаляются квадратные скобки, и строка разбивается на массив отдельных значений;
- из поля с адресом извлекаются округ, район, полный адрес и четыре параметра доступной среды;
- из поля с географическими данными извлекаются широта и долгота;
- глобальный идентификатор преобразуется в целочисленный тип для обеспечения уникальности записей.

## Архитектура приложения

Разработанное веб-приложение построено по классической клиент-серверной архитектуре с использованием шаблона MVC (Model-View-Controller) в интерпретации, принятой во фреймворке Flask. Клиентская часть представлена веб-браузером, который отображает HTML-страницы и обрабатывает действия пользователя. Серверная часть реализована на языке Python с использованием микрофреймворка Flask и выполняет обработку HTTP-запросов, взаимодействие с базой данных и формирование ответов.

Структура проекта организована следующим образом:

- `app.py` — основной файл приложения, содержащий все маршруты и бизнес-логику;
- `load_data.py` — отдельный скрипт для первичной загрузки и предобработки данных из XLSX в PostgreSQL;
- `templates/` — папка с HTML-шаблонами для каждой страницы приложения;
- `sport_data.xlsx` — исходный датасет, полученный с портала открытых данных;

При обработке запроса выполняются следующие этапы:

1. Маршрутизация. Flask анализирует URL запроса и вызывает соответствующую функцию (view function). Например, запрос к корневому URL '/' обрабатывается функцией `index()`, а запрос вида '/facility/123' — функцией `facility_detail(123)`.
2. Формирование параметров. Из запроса извлекаются параметры: для GET-запросов они берутся из строки URL (например, фильтры поиска), для POST-запросов — из тела запроса (например, данные формы регистрации).
3. Взаимодействие с базой данных. Функция формирует SQL-запрос с использованием параметров, выполняет его через библиотеку psycopg2 и получает результаты в виде словарей или списков.
4. Бизнес-логика. При необходимости выполняется дополнительная обработка полученных данных. Например, для геопоиска вычисляется расстояние до объектов, для отображения часов работы удаляются служебные слова DayWeek и WorkHours.
5. Формирование ответа. Подготовленные данные передаются в HTML-шаблон с помощью функции `render_template`. Flask выполняет подстановку переменных в шаблон и возвращает готовую HTML-страницу клиенту.

Такая архитектура обеспечивает чёткое разделение ответственности между компонентами: шаблоны отвечают только за отображение, функции-обработчики — за логику, а база данных - за хранение и извлечение информации. Это упрощает разработку, тестирование и дальнейшее сопровождение приложения.

Взаимодействие с базой данных организовано через `get_db_connection()` - отдельную функцию , которая создаёт подключение к PostgreSQL с фиксированными параметрами (хост localhost, база данных sport_ryadom, пользователь user). Для выполнения запросов используется курсор с расширением `DictCursor`, позволяющий обращаться к полям записи по именам, что делает код более читаемым по сравнению с обращением по индексам.

Для каждого запроса создаётся новое подключение к базе данных, которое закрывается после выполнения всех операций. Такой подход не создаёт избыточной нагрузки благодаря быстрому установлению соединений с локальной базой данных и гарантирует, что ресурсы освобождаются даже в случае возникновения ошибок.

## Фильтрация и поиск

Фильтрация объектов реализована через динамическое формирование SQL-запроса в функции index. В зависимости от переданных параметров к базовому запросу добавляются соответствующие условия. Для поиска по названию используется оператор ILIKE, обеспечивающий регистронезависимый поиск с поддержкой частичного совпадения. Для фильтрации по виду спорта применяется конструкция ANY(services), проверяющая вхождение значения в массив PostgreSQL. Фильтр по доступности реализован через точное сравнение значения поля available_k.

В листинге 1 представлена функция фильтрации объектов.

Листинг 1 : Функция фильтрации объектов

```
def index():
    conn = get_db_connection()
    cur = conn.cursor(cursor_factory=psycopg2.extras.DictCursor)

    sport_filter = request.args.get('sport', '')
    available_filter = request.args.get('available', '')
    name_filter = request.args.get('name', '')

    query = "SELECT * FROM sport_facilities WHERE 1=1"
    params = []

    if name_filter:
        query += " AND name ILIKE %s"
        params.append(f'%{name_filter}%')

    if sport_filter:
        query += " AND %s = ANY(services)"
        params.append(sport_filter)

    cur.execute(query, params)
    facilities = cur.fetchall()
    cur.close()
    conn.close()

    return render_template('index.html', facilities=facilities)
```

Детальная страница объекта реализована в маршруте facility/<int:id>. При обращении к этому маршруту выполняется запрос к базе данных для получения полной информации об объекте. Для зарегистрированных пользователей дополнительно выполняется проверка наличия данного объекта в избранном. На странице отображаются все характеристики объекта, включая адрес, часы работы, контакты, виды спорта и параметры доступной среды. Также на страницу встраивается карта OpenStreetMap с меткой местоположения объекта.

## Аутентификация и безопасность

Система аутентификации реализована с использованием библиотеки Flask-Login. Пользователи могут регистрироваться, указывая имя, фамилию, email и пароль. Пароли хранятся в базе данных в виде хэш-сумм, создаваемых с помощью функции generate_password_hash из библиотеки Werkzeug.

В листинге 2 представлена функция регистрации, демонстрирующая хэширование пароля перед сохранением в базу данных.

Листинг 2 : Регистрация пользователя

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        first_name = request.form['first_name']
        last_name = request.form['last_name']
        email = request.form['email']
        password = request.form['password']

        password_hash = generate_password_hash(password)

        cur.execute("""
            INSERT INTO users (first_name, last_name, email, password_hash)
            VALUES (%s, %s, %s, %s)
        """, (first_name, last_name, email, password_hash))
        conn.commit()

        return redirect(url_for('login'))
```

При входе введённый пароль проверяется функцией check_password_hash. Для доступа к некоторым маршрутам (например, к личному кабинету) требуется аутентификация, что обеспечивается декоратором @login_required.

При работе с базой данных используются параметризованные запросы (подстановка значений через %s), что полностью исключает возможность SQL-инъекций. Все входящие данные от пользователя проходят через механизм параметризации, гарантирующий, что они не будут интерпретироваться как часть SQL-кода.

## Избранное и личный кабинет

Функционал избранного реализован через маршрут `toggle_favorite`, принимающий POST-запросы. При добавлении объекта в избранное создаётся запись в таблице `favorites`, связывающая идентификатор пользователя с идентификатором объекта. При удалении соответствующая запись удаляется. После выполнения действия пользователь возвращается на предыдущую страницу с помощью редиректа.

В листинге 3 показан SQL-запрос для получения списка избранных объектов пользователя.

Листинг 3 : Получение списка избранного

```
cur.execute("""
    SELECT f.*, sf.name, sf.address
    FROM favorites f
    JOIN sport_facilities sf ON f.facility_id = sf.id
    WHERE f.user_id = %s
    ORDER BY f.created_at DESC
""", (current_user.id,))

favorites = cur.fetchall()
```

Личный кабинет доступен по маршруту `profile` и содержит информацию о пользователе, форму для ввода домашнего адреса и список избранных объектов.

## Сравнение объектов

Сравнение объектов реализовано через маршрут `compare`. В листинге 4 представлена функция сравнения, реализующая сложную логику выбора.

Листинг 4 : Функция сравнения объектов

```
@app.route('/compare')
def compare():
    ids = request.args.getlist('ids')
    int_ids = [int(id) for id in ids if id.isdigit()]

    conn = get_db_connection()
    cur = conn.cursor(cursor_factory=psycopg2.extras.DictCursor)

    facilities = []
    for id in int_ids:
        cur.execute("SELECT * FROM sport_facilities WHERE id = %s", (id,))
        facility = cur.fetchone()
        if facility:
            facilities.append(facility)

    if len(facilities) > 3:
        session['original_ids'] = int_ids
        cur.close()
        conn.close()
        return render_template('select_compare.html', facilities=facilities)

    if len(facilities) < 2:
        flash('Please select at least 2 items to compare.', 'error')
        cur.close()
        conn.close()
        return redirect(url_for('index'))

    cur.close()
    conn.close()
    return render_template('compare.html', facilities=facilities)
```

При выборе объектов на главной странице их идентификаторы передаются в параметрах запроса. Если выбрано менее двух объектов, пользователю показывается сообщение об ошибке. Если выбрано более трёх объектов, открывается страница выбора `select_compare`, где пользователь может снять лишние объекты.

При выборе от двух до трёх объектов открывается страница сравнения `compare` с таблицей, содержащей характеристики выбранных объектов.

### Геопоиск

Геопоиск реализован в маршруте nearby. При обращении к этому маршруту проверяется наличие домашнего адреса у текущего пользователя. Если адрес указан, он преобразуется в географические координаты с помощью API Nominatim (OpenStreetMap).

Листинг 5 : Функция геокодинга

```
def get_coordinates_from_address(address):
    if not address:
        return None, None

    from urllib.parse import quote
    encoded_address = quote(address)
    url = f"https://nominatim.openstreetmap.org/search?q={encoded_address}&format=
   json&limit=1"

    headers = {
        'User-Agent': 'SportRyadomApp/1.0',
        'Accept-Language': 'ru'
    }

    try:
        response = requests.get(url, headers=headers)
        response.encoding = 'utf-8'
        data = response.json()

        if data and len(data) > 0:
            return float(data[0]['lat']), float(data[0]['lon'])
    except Exception as e:
        print(f"Geocoding error: {e}")

    return None, None
```

Затем из базы данных извлекаются все объекты с координатами, и для каждого объекта вычисляется расстояние до указанного адреса по формуле гаверсинуса. Полученный список сортируется по расстоянию, и первые десять объектов отображаются на странице.

Для геокодинга используется открытый сервис Nominatim, не требующий регистрации и получения ключа API. В запросе к сервису обязательно указывается User-Agent для идентификации приложения. При возникновении ошибок или отсутствии результатов пользователю показывается соответствующее сообщение.

## Основные сценарии использования

Рассмотрим основные сценарии использования разработанного приложения с точки зрения пользователя. Каждый сценарий описывает последовательность действий пользова-

теля и реакцию системы.

## Сценарий 1: Поиск объекта с фильтрами

Пользователь открывает главную страницу приложения. В верхней части страницы расположена панель фильтров, содержащая следующие элементы:

- поле ввода для поиска по названию объекта;
- выпадающий список для выбора вида спорта (содержит все уникальные виды спорта из базы данных);
- выпадающий список для выбора доступности для колясочников (значения: «Любая», «Полностью доступно», «Частично доступно», «Недоступно»);
- выпадающий список для выбора района Москвы;
- кнопка «Применить фильтры».

Результат поиска с применёнными фильтрами показан на рисунке 5. На главной странице отображаются карточки объектов, удовлетворяющих заданным критериям.

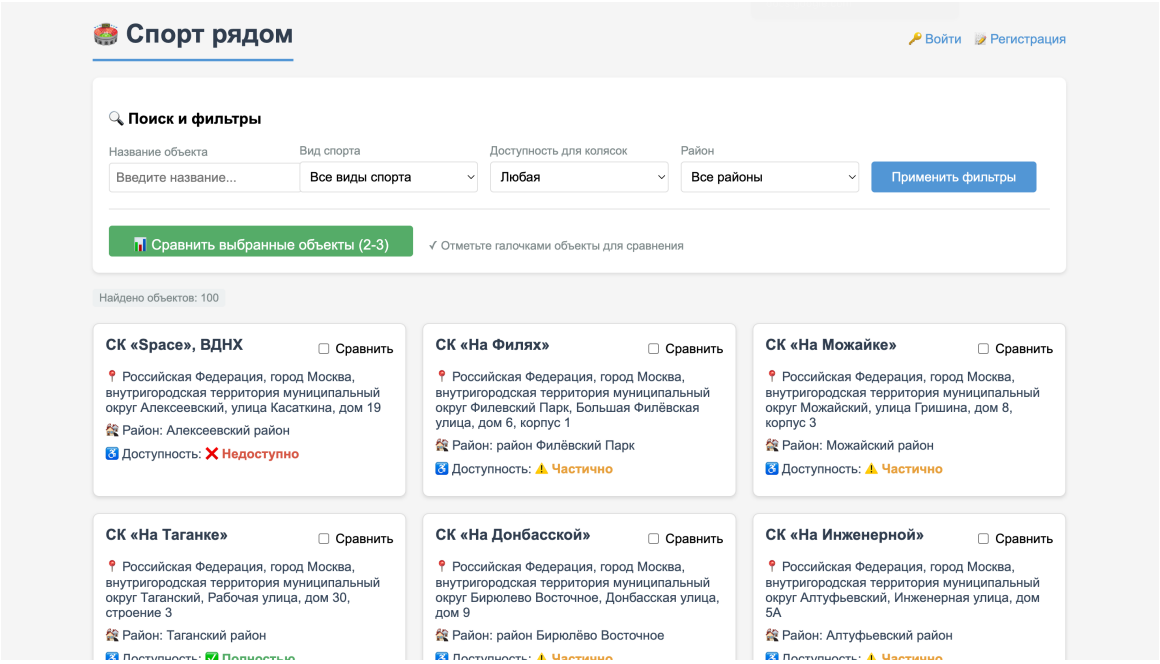


Рис. 5: Главная страница с результатами поиска

Пользователь может комбинировать фильтры произвольным образом. Например, он может ввести в поле поиска «Куркино», выбрать вид спорта «Футбол» и доступность «Частично доступно». После нажатия кнопки «Применить фильтры» страница перезагружается, и приложение формирует SQL-запрос к базе данных, учитывающий все выбранные критерии.

В верхней части списка результатов отображается индикатор с количеством найденных объектов. Если фильтры применены, рядом с индикатором появляется дополнительная метка «Фильтры применены». Каждый объект представлен в виде карточки, содержащей название (со ссылкой на детальную страницу), адрес, район и информацию о доступности с соответствующим цветовым обозначением (зелёный — полностью доступно, жёлтый — частично доступно, красный — недоступно).

Если ни один объект не удовлетворяет заданным критериям, отображается сообщение «Ничего не найдено. Попробуйте изменить фильтры».

## Сценарий 2: Просмотр детальной информации

На главной странице пользователь видит список объектов в виде карточек. Заинтересовавшись конкретным объектом, пользователь нажимает на его название и переходит на детальную страницу.

Детальная страница содержит следующие блоки информации:

- Заголовок с названием объекта. Для авторизованных пользователей справа от названия отображается кнопка в виде сердечка для добавления в избранное.
- Блок «Расположение» с точным адресом, административным округом и районом.
- Блок «Виды спорта» — все доступные на объекте виды спорта отображаются в виде цветных тегов. Это позволяет быстро оценить, подходит ли объект для интересующего пользователя вида спорта.
- Блок «Доступная среда» — для каждой из четырёх категорий (колясочники, слабовидящие, слабослышащие, ментальные особенности) указан уровень доступности с соответствующим цветовым обозначением.
- Блок «Часы работы» — график работы представлен в виде пар «день: время», при этом из исходных данных удалены служебные слова DayWeek и WorkHours.
- Блок «Контакты» — телефон, email и ссылка на сайт (если доступны). Номера телефонов автоматически форматируются с добавлением кода страны +7.
- Блок «Карта» — интерактивная карта OpenStreetMap с меткой местоположения объекта. Под картой расположена ссылка для открытия карты в полном окне.

Детальная страница объекта разделена на две части для удобства просмотра. На рисунке 6 показана верхняя часть страницы с названием объекта, видами спорта и параметрами доступной среды. На рисунке 7 представлена нижняя часть страницы с часами работы, контактными данными и картой.

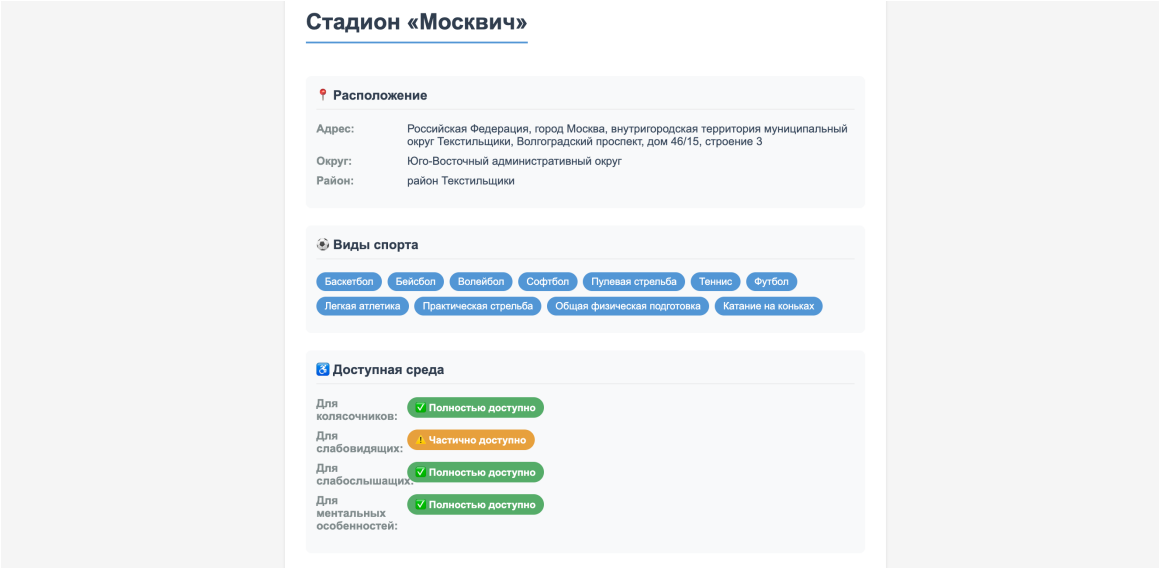


Рис. 6: Детальная страница (виды спорта и доступная среда)

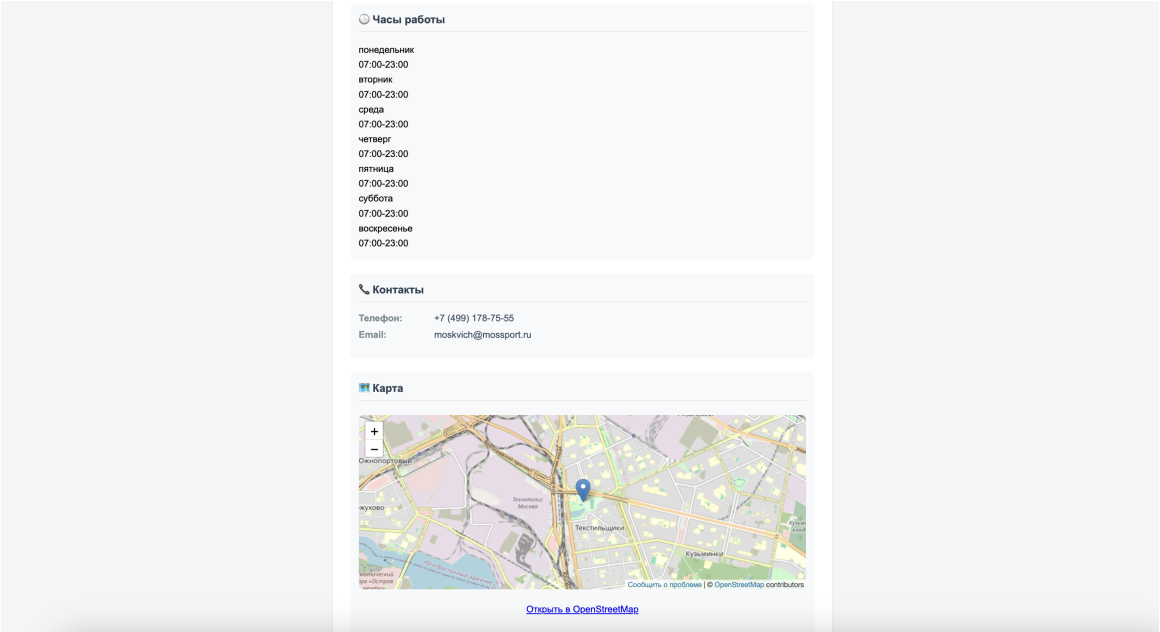


Рис. 7: Детальная страница (часы работы, контакты, карта)

Все данные на странице представлены в структурированном и легко читаемом виде, что позволяет пользователю быстро получить полную информацию об объекте.

## Сценарий 3: Добавление в избранное

Для использования функционала избранного пользователь должен быть авторизован в системе. На детальной странице объекта справа от названия расположена кнопка в виде сердечка, которая индицирует текущий статус объекта:

- белое сердечко — объект не в избранном;
- красное сердечко — объект уже добавлен в избранное.

Личный кабинет пользователя представлен на рисунке 8. На странице отображаются имя и фамилия пользователя, email, форма для ввода домашнего адреса и список избранных объектов.

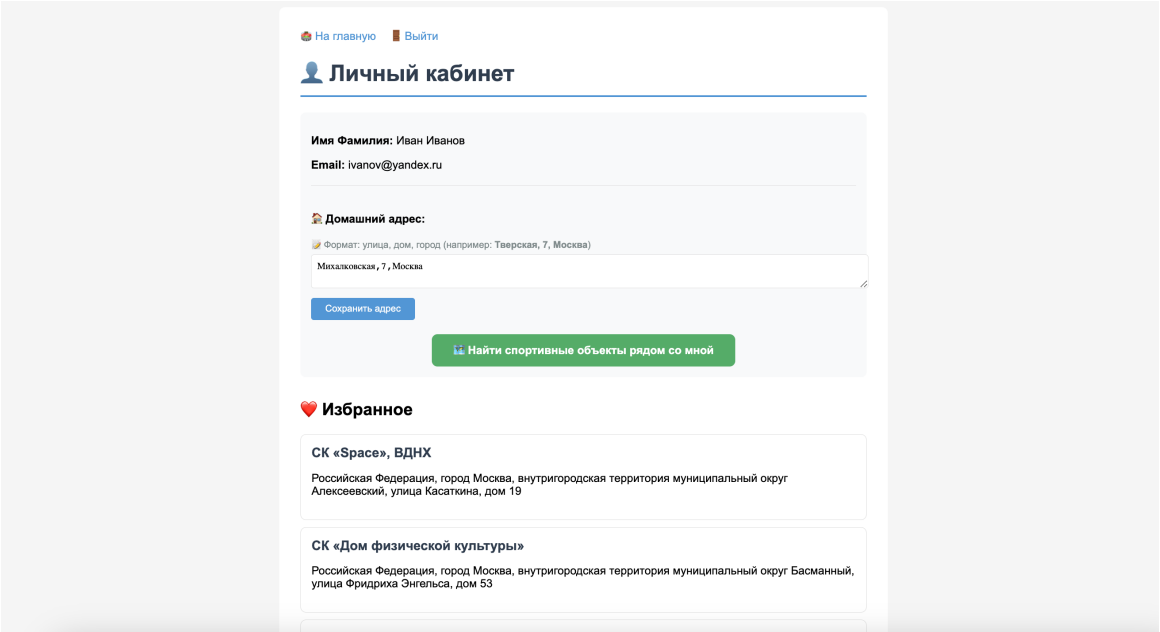


Рис. 8: Личный кабинет пользователя

При нажатии на белое сердечко выполняется POST-запрос к серверу. Сервер проверяет, есть ли уже запись в таблице favorites для данного пользователя и объекта. Если записи нет, она создаётся. После выполнения операции страница перезагружается, сердечко становится красным, и в верхней части страницы появляется зелёное сообщение «Добавлено в избранное».

При нажатии на красное сердечко выполняется аналогичный запрос, но запись из таблицы favorites удаляется. После перезагрузки страницы сердечко становится белым, и показывается сообщение «Удалено из избранного».

Таким образом, пользователь получает немедленную обратную связь о результате своего действия.

## Сценарий 4: Просмотр избранного

Авторизованный пользователь может перейти в личный кабинет, нажав на своё имя в правом верхнем углу любой страницы. Личный кабинет содержит следующие элементы:

- приветствие с именем и фамилией пользователя;
- email пользователя;
- форма для ввода и сохранения домашнего адреса (с подсказкой о формате ввода);
- кнопка «Найти спортивные объекты рядом со мной» (появляется после сохранения адреса);
- список избранных объектов.

Список избранного формируется SQL-запросом, объединяющим таблицы favorites и sport_facilities. Для каждого объекта отображается его название (в виде ссылки на детальную страницу) и адрес. Если избранное пусто, вместо списка отображается сообщение «У вас пока нет избранных объектов».

Из личного кабинета пользователь может быстро перейти к любому избранному объекту или удалить объекты из избранного, перейдя на их детальные страницы.

## Сценарий 5: Сравнение объектов

На главной странице у каждой карточки объекта имеется флажок «Сравнить». Пользователь может отметить флажки у нескольких объектов для их последующего сравнения.

Логика сравнения реализована следующим образом:

- Если отмечен **один** объект и нажата кнопка «Сравнить выбранные объекты», показывается сообщение об ошибке: «Выберите хотя бы 2 объекта для сравнения».
- Если отмечено **два или три** объекта, открывается страница сравнения, где характеристики объектов представлены в виде таблицы. Строки таблицы сгруппированы по типам характеристик: адрес, округ, район, виды спорта, доступная среда (по четырём категориям), часы работы, телефон, email, сайт, карта. Для видов спорта используется компактное отображение в виде тегов.
- Если отмечено **более трёх** объектов, открывается промежуточная страница выбора `select_compare`. На этой странице все выбранные объекты отображаются с уже установленными флажками, и пользователь может снять лишние, оставив от двух до трёх для сравнения. При отправке формы серверная логика на Python проверяет количество выбранных объектов: если выбрано менее двух, пользователь возвращается на страницу выбора с сообщением об ошибке; если выбрано два или три — открывается страница сравнения.

Страница сравнения трёх объектов показана на рисунке 9. Характеристики объектов представлены в виде таблицы, что позволяет наглядно сопоставить их по всем параметрам.

Рис. 9: Сравнение трёх спортивных объектов

Страница сравнения предоставляет удобный способ сопоставить ключевые характеристики нескольких объектов и принять обоснованное решение.

## Сценарий 6: Поиск ближайших объектов

Для использования геопоиска пользователь должен быть авторизован и указать свой домашний адрес в личном кабинете. Форма ввода адреса содержит подсказку с примером

правильного формата: «Тверская, 7, Москва». После ввода адреса и нажатия кнопки «Сохранить адрес» данные сохраняются в таблице users, и под формой появляется кнопка «Найти спортивные объекты рядом со мной».

При нажатии на эту кнопку выполняются следующие шаги:

1. Приложение отправляет запрос к API Nominatim (OpenStreetMap) с адресом пользователя. Для корректной обработки русских символов адрес кодируется, и в заголовках указывается User-Agent.

2. API возвращает географические координаты (широту и долготу) для указанного адреса. Если адрес не найден, пользователю показывается сообщение «Не удалось определить координаты вашего адреса» с предложением проверить правильность ввода.

3. Из базы данных извлекаются все спортивные объекты, для которых указаны координаты (поля latitude и longitude не равны NULL).

4. Для каждого объекта вычисляется расстояние до адреса пользователя по формуле гаверсинуса, которая учитывает сферическую форму Земли:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta lat}{2}\right) + \cos(lat_1)\cos(lat_2)\sin^2\left(\frac{\Delta lon}{2}\right)}\right)$$

   где $R = 6371$ км — радиус Земли.

   Хотя для небольших расстояний (до 5–10 км) можно было бы использовать упрощённые формулы, например теорему Пифагора в прямоугольных координатах, применение формулы гаверсинуса обусловлено стремлением обеспечить корректность вычислений для всех возможных расстояний и придать работе научно-теоретическую обоснованность. Это также демонстрирует понимание особенностей геометрических расчётов на сферической поверхности.

5. Полученный список сортируется по возрастанию расстояния, и первые десять объектов отображаются на странице результатов.

Результаты поиска ближайших объектов к домашнему адресу пользователя представлены на рисунке 10. Для каждого объекта указано расстояние в километрах, название, адрес и информация о доступности.

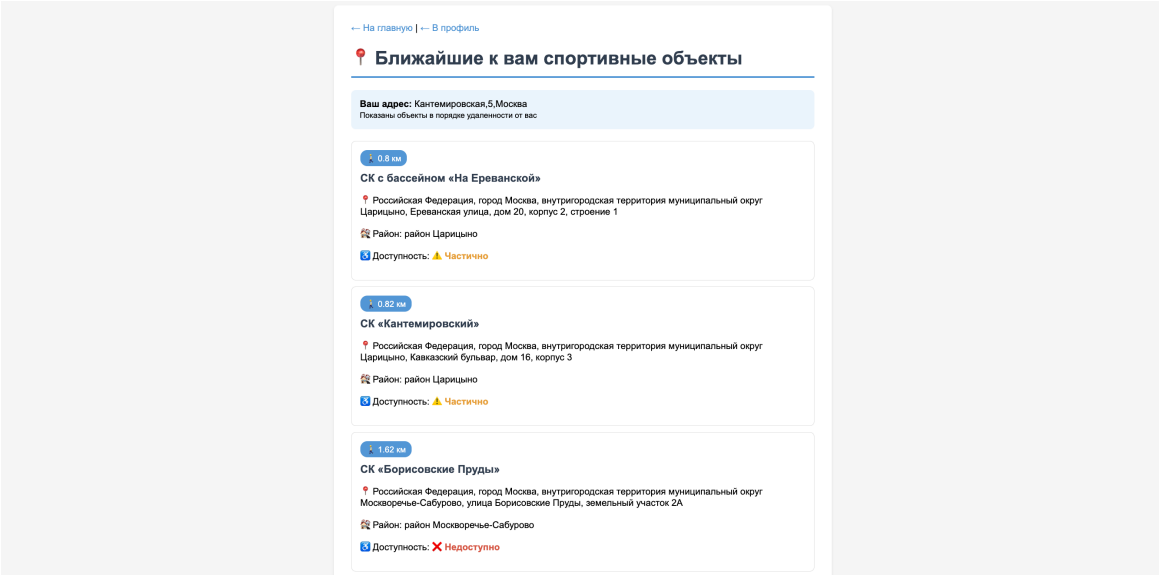


Рис. 10: Список объектов, ближайших к дому пользователя

На странице результатов для каждого объекта указывается расстояние в километрах, название (со ссылкой на детальную страницу), адрес, район и информация о доступности. Это позволяет пользователю быстро оценить, какие объекты находятся ближе всего к его дому.

Если в базе данных нет объектов с координатами или ни один объект не найден в пределах разумного радиуса, отображается соответствующее сообщение.

### Сценарий 7: Регистрация и вход в систему

Для доступа к персонализированным функциям (избранное, геопоиск) пользователь должен зарегистрироваться или войти в систему.

**Регистрация:** Пользователь нажимает ссылку «Регистрация» в правом верхнем углу и заполняет форму, содержащую поля: имя, фамилия, email, пароль. При отправке формы пароль хэшируется с помощью функции generate_password_hash из библиотеки Werkzeug, после чего данные сохраняются в таблице users. Если указанный email уже существует, показывается сообщение об ошибке. После успешной регистрации пользователь автоматически входит в систему и перенаправляется в личный кабинет.

**Вход:** Пользователь нажимает ссылку «Войти» и вводит email и пароль. Сервер проверяет наличие пользователя с таким email и сравнивает хэш введённого пароля с сохранённым в базе данных с помощью функции check_password_hash. При успешной проверке создаётся пользовательская сессия с помощью Flask-Login, и пользователь перенаправляется на главную страницу. При неверном email или пароле показывается сообщение об ошибке.

**Выход:** Авторизованный пользователь может выйти из системы, нажав ссылку «Выйти» в правом верхнем углу. Сессия завершается, и пользователь перенаправляется на главную страницу.

## Заключение

В ходе выполнения курсового проекта было разработано веб-приложение «Спорт рядом», предназначенное для поиска спортивных объектов Москвы. Приложение реализует все

заявленные функции, соответствующие пользовательским историям, сформулированным на этапе проектирования.

В результате выполнения работы были достигнуты следующие результаты:

- проведён анализ открытых источников данных и выбран датасет «Спортивные объекты города Москвы» с портала data.mos.ru, содержащий 810 записей;
- выполнена предобработка и очистка данных: из вложенных структур извлечены параметры доступной среды, координаты, часы работы; удалены служебные слова DayWeek и WorkHours; выполнено форматирование телефонных номеров;
- спроектирована структура базы данных PostgreSQL, включающая следующие три таблицы: sport_facilities, users, favorites;
- разработана серверная часть на языке Python с использованием фреймворка Flask, реализующая обработку запросов и бизнес-логику;
- создан пользовательский интерфейс с помощью HTML и CSS, включающий макеты всех основных страниц;
- реализован функционал фильтрации объектов по названию, виду спорта, району и доступности с использованием динамических SQL-запросов;
- реализована система аутентификации с хэшированием паролей и защитой маршрутов;
- реализован функционал избранного с возможностью добавления и удаления объектов;
- реализован функционал сравнения объектов с умной логикой выбора (не менее двух, не более трёх);
- реализован геопоиск с использованием API Nominatim и вычислением расстояния по формуле гаверсинуса;
- выполнено тестирование и отладка приложения, все функции работают в соответствии с требованиями.

Разработанное приложение соответствует критериям :

- структура приложения многосложна и включает множество взаимосвязанных модулей;
- интерфейс эргономичен и интуитивно понятен;
- веб-страницы динамические и интерактивные;
- реализована система регистрации и авторизации пользователей;
- проработаны аспекты защиты информации: хэширование паролей, параметризованные запросы для защиты от SQL-инъекций, управление доступом через Flask-Login;
- для поддержания функционала приложения используется база данных PostgreSQL;
- реализована форма поиска с фильтрацией по нескольким критериям;

- использованы средства визуализации данных: карты OpenStreetMap, цветовые индикаторы доступности, теги видов спорта.

Приложение может быть использовано жителями Москвы для выбора подходящих мест для занятий спортом с учётом их индивидуальных предпочтений и потребностей. Оно обеспечивает удобный доступ к открытым государственным данным и способствует повышению осведомлённости граждан о доступных спортивных объектах, в том числе для маломобильных групп населения.

В дальнейшем возможно расширение функционала приложения по следующим направлениям:

- добавление возможности оставлять отзывы и оценки для объектов с рейтингованием;
- интеграция с дополнительными источниками данных, например, со станциями метро для более точного геопоиска;
- разработка мобильной версии или нативного мобильного приложения;
- реализация автоматического обновления данных при изменении датасета на портале data.mos.ru;
- добавление рекомендательной системы на основе предпочтений пользователя и истории просмотров;
- улучшение дизайна с использованием современных CSS-фреймворков.

Исходный код проекта доступен в репозитории GitHub по адресу: `https://github.com/Stepa-do-it/sport_ryadom`

# Список литературы


1. Портал открытых данных Правительства Москвы [Электронный ресурс]. – Режим доступа: `https://data.mos.ru/` (дата обращения: 16.12.2025).
2. Документация Flask [Электронный ресурс]. – Режим доступа: `https://flask.palletsprojects.com/` (дата обращения: 10.02.2026).
3. Документация PostgreSQL [Электронный ресурс]. – Режим доступа: `https://www.postgresql.org/docs/` (дата обращения: 01.02.2026).
4. Документация Nominatim [Электронный ресурс]. – Режим доступа: `https://nominatim.org/release-docs/develop/` (дата обращения: 12.02.2026).
5. Документация Python 3.11 [Электронный ресурс]. – Режим доступа: `https://docs.python.org/3.11/` (дата обращения: 01.02.2026).
6. Документация Psycopg2 [Электронный ресурс]. – Режим доступа: `https://www.psycopg.org/docs/` (дата обращения: 12.02.2026).
7. Документация Flask-Login [Электронный ресурс]. – Режим доступа: `https://flask-login.readthedocs.io/` (дата обращения: 12.02.2026).
8. Документация Werkzeug [Электронный ресурс]. – Режим доступа: `https://werkzeug.palletsprojects.com/` (дата обращения: 12.02.2026).

9. Документация Pandas [Электронный ресурс]. – Режим доступа: `https://pandas.pydata.org/docs/` (дата обращения: 08.02.2026).

10. OpenStreetMap Nominatim [Электронный ресурс]. – Режим доступа: `https://nominatim.org/` (дата обращения: 14.02.2026).

11. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. – М.: Стандартинформ, 2008. – 20 с.

12. Lutz M. Программирование на Python. – М.: Символ-Плюс, 2020. – 1152 с.

13. Grinberg M. Flask Web Development. – O'Reilly Media, 2018. – 314 с.